# Granja concurrente de impresión 3D

Trabajo Práctico 1 — Programación Concurrente y Paralela 2026

Rafael Barrio  Alexis Tomás Garay  Pedro Guzman Gonzalez  Santino Chesta

Valentino Zucchella Paz

Grupo Warrior-of-Interruptions — 2026

## Integrantes y repositorio

| Nombre | Documento | Usuario Git |
|---|---|---|
| Rafael Barrio | 46472542 | Rafa-Barrio |
| Alexis Tomás Garay | 46032555 | Garay1913 |
| Pedro Guzman Gonzalez | 46068277 | PedroGuzmanGonzalez |
| Santino Chesta | 45711131 | SantinoChesta |
| Valentino Zucchella Paz | 44578880 | ValentinoZucchellaPaz |

- **Grupo:** Warrior-of-Interruptions
- **Repositorio:** https://github.com/ValentinoZucchellaPaz/tp1-pcyp-2026.git
- **Tag de entrega:** `Warrior-of-Interruptions-entrega-tp1`

## 1. Introducción y objetivo

El trabajo simula una granja industrial de impresión 3D. El sistema recibe órdenes de fabricación, les asigna una impresora, valida el modelo, ejecuta la impresión y controla la calidad de la pieza obtenida. Cada orden comienza en `CREATED` y finaliza exactamente en uno de cuatro estados terminales: `APPROVED`, `REJECTED`, `PRINT_FAILED` o `DEFECTIVE`.

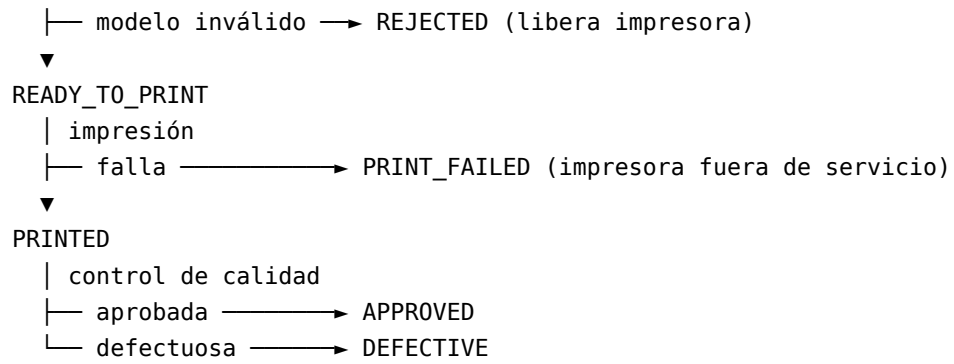
Los objetivos son:

i. que las cuatro etapas funcionen **concurrentemente**
ii. que compartan recursos de manera segura
iii. que todas las órdenes se procesen conservando la consistencia de sus estados y
iv. que la simulación finalice sin dejar hilos activos ni impresoras reservadas.

## 2. Arquitectura general

La solución organiza el trabajo como un **pipeline de cuatro etapas** comunicadas por colas FIFO. El hilo principal (`main`) crea las órdenes y las deposita en la primera cola; a partir de ahí, los workers avanzan cada orden por el recorrido:

```
CREATED
  | asignación: reserva una impresora
  ▼
WAITING_VALIDATION
  | validación
```




La cantidad de workers por etapa se toma de la configuración (config oficial: 3 asignación, 2 validación, 3 impresión, 2 calidad). **Todas las etapas se inician al comienzo**: cada worker se registra y espera en una compuerta común (`StartGate`) hasta que el principal confirma que los diez están listos y la abre. De este modo ninguna etapa procesa una orden antes de que todas hayan arrancado.

## 2.1 Diagrama de clases

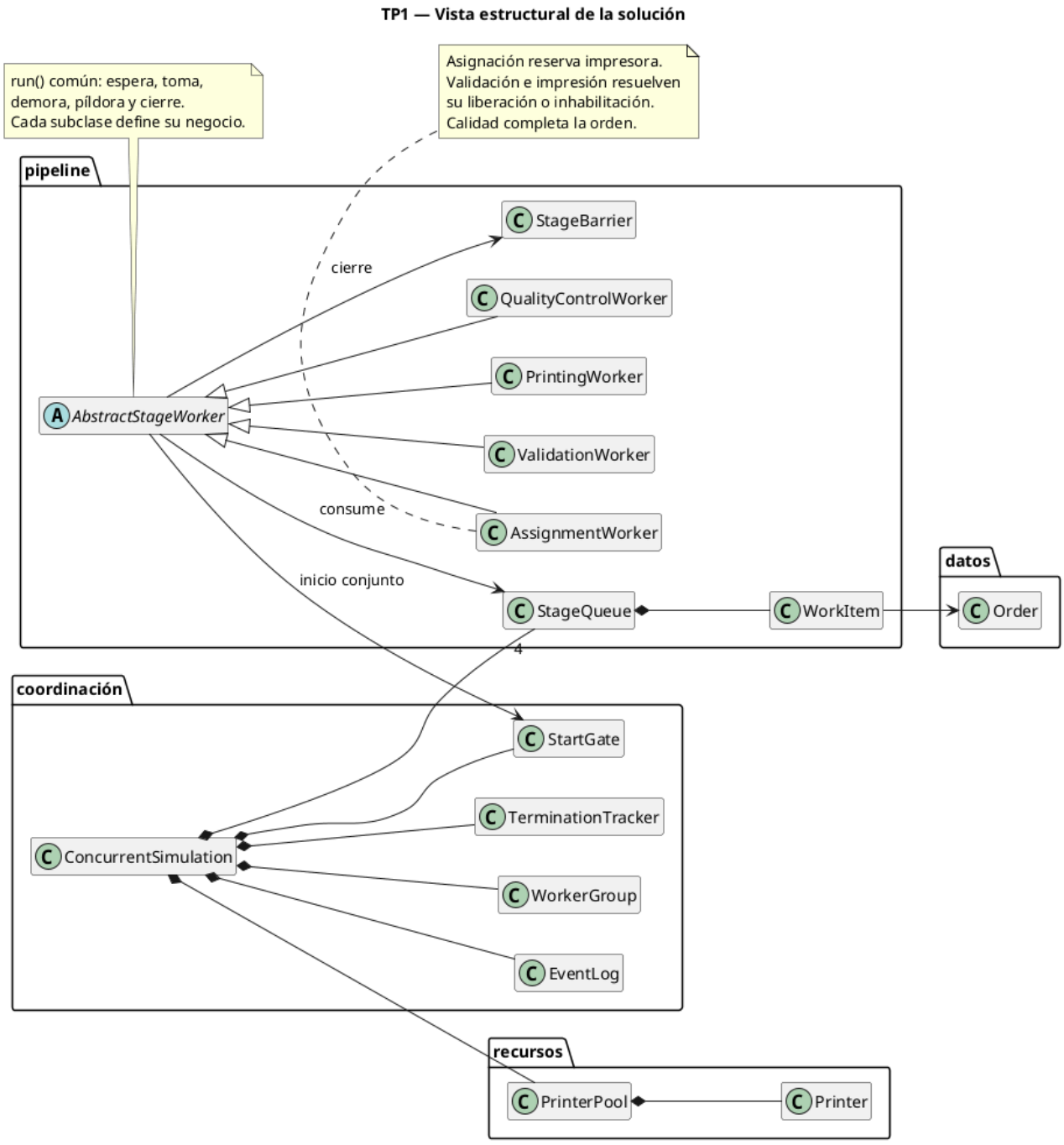



Figure 1: Diagrama de clases simplificado de la solución

`ConcurrentSimulation` es el punto de entrada: crea las órdenes y las impresoras, las cuatro `StageQueue` y los diez workers.
Los cuatro workers de etapa (`AssignmentWorker`, `ValidationWorker`, `PrintingWorker`, `QualityControl-Worker`) heredan de `AbstractStageWorker`, que concentra el protocolo común (retirar de la cola, reconocer píldoras, reportar al `StageBarrier`), cada implementacion cambia la acción del hilo para procesar y cuando termina.
`WorkItem` envuelve el trabajo que circula entre colas (una orden o una píldora) y `StageQueue` es la frontera FIFO entre etapas.
`PrinterPool` administra la matriz de impresoras.
Las clases de `runtime` (`StartGate`, `StageBarrier`, `TerminationTracker`, `WorkerGroup`) coordinan arranque,

cierre y terminación.

## 2.2 Diagrama de secuencia

La vida del programa se separa en 4 diagramas de secuencia para mejor entendimiento de este:

1. Arranque: inicializacion de workers e inicio coordinado
2. Recorrido: vida y camino feliz de cada worker
3. Salidas anticipadas: no realizan todas las etapas
4. Cierre: estrategia para terminar workers e hilo principal (poison pills).

La secuencia comienza con el **arranque conjunto**: los diez workers se registran en se da arranque (`workers.startAll()`). Quedan esperando en `StartGate` hasta que `main` abre la compuerta; así ninguna etapa procesa una orden antes de que todas las etapas estén iniciadas. Los hilos aumentan un contador, luego notifican a todos y se van a dormir, llega un momento que al levantar todos, el main chequea la condicion y allí levanta la barrera.

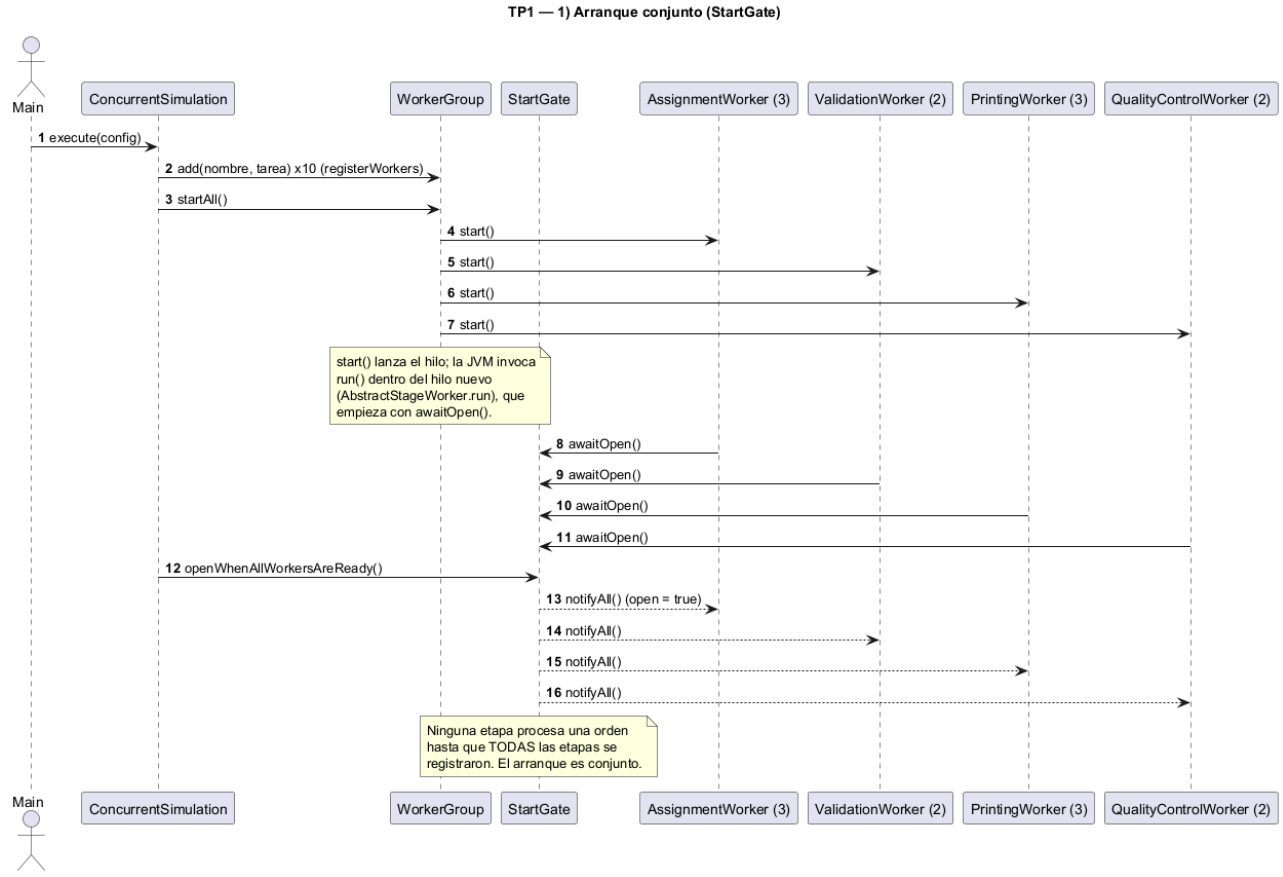



Figure 2: Diagrama de secuencia — arranque

Luego se sigue el **recorrido feliz** de una orden: **asignación** (reserva una impresora): de CREATED a WAITING_VALIDATION -> **validacion**: de WAITING_VALIDATION a READY_TO_PRINT → **impresión** (libera impresora): de READY_TO_PRINT a PRINTED → **calidad**: de PRINTED a APPROVED. Cuando terminan de procesar una orden correctamente, aumentan el contador de `TerminationTracker` y ofrecen la orden a la cola de salida (entrada de la próxima etapa). En cada transición el worker registra el evento en `EventLog` —asignando el `sequence`— antes de encolar la orden en la etapa siguiente.

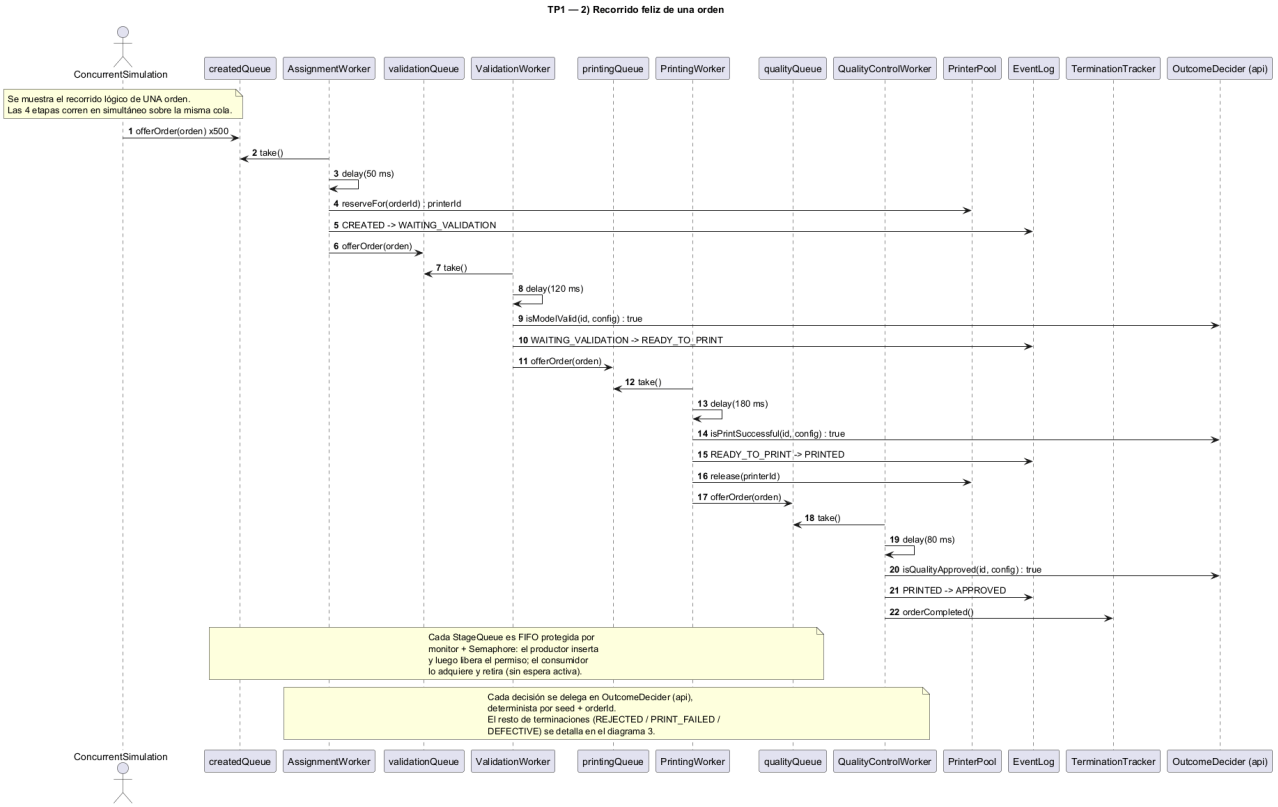



Figure 3: Diagrama de secuencia — recorrido

El diagrama incluye las **salidas anticipadas** que se dan cuando OutcomeDecider devuelve false:

- en `validation`: modelo inválido → `REJECTED` (y libera la impresora -> `AVAILABLE`)
- en `printing`: fallo de impresión → `PRINT_FAILED` (y marca impresora como `OUT_OF_SERVICE`)
- en `quality control`: fallo en control de calida -> `REJECTED` (y libera la impresora -> `AVAILABLE`). Notar que sigue informando a `TerminationTracker` cuando una orden se completa.

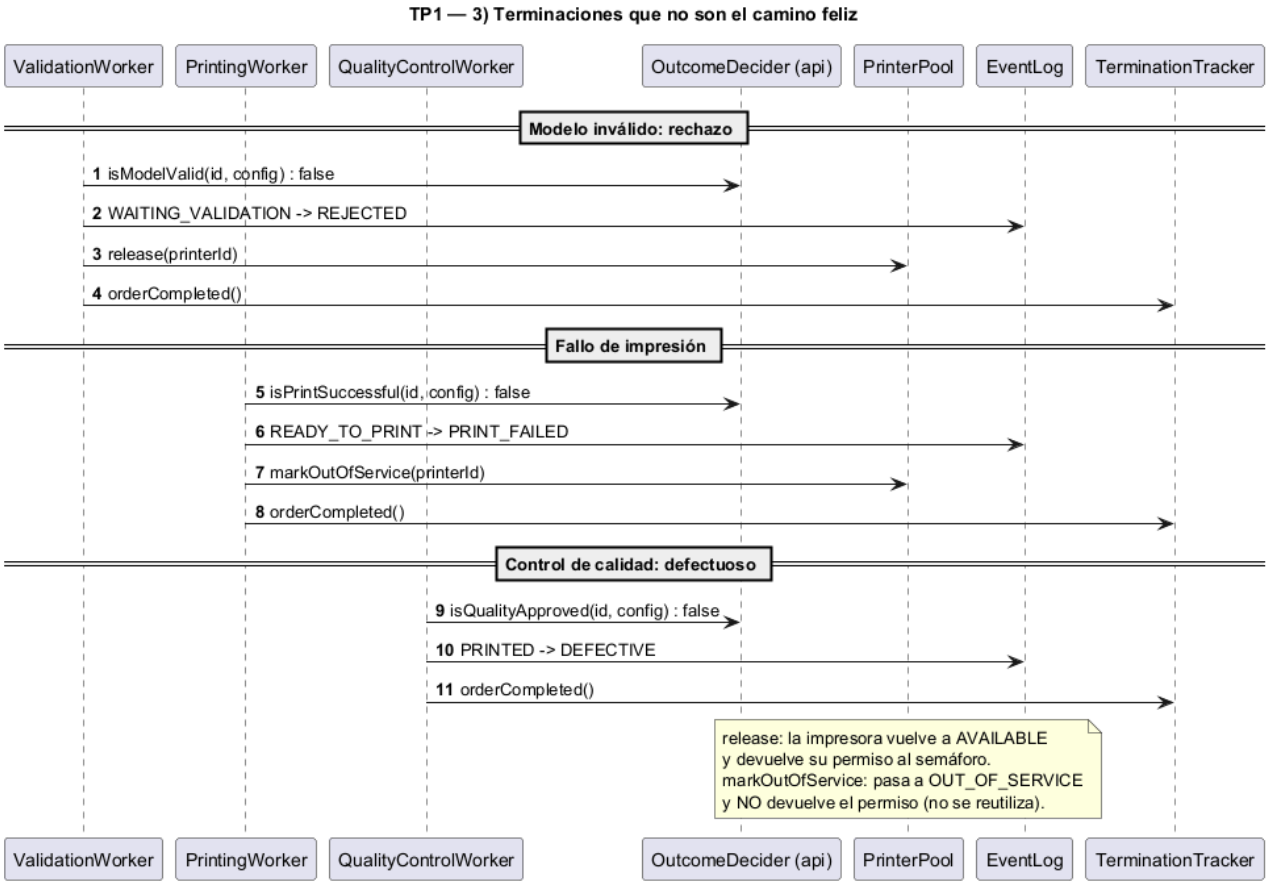



Figure 4: Diagrama de secuencia — salidas

La terminacion en cascada de los hilos se resuelve con el paradigma de pildoras venenosas. Las colas de entrada y salida `StageQueue` contienen `WorkItem`, que puede ser `Order` o `null` (pildora) Cuando un worker toma una pildora, es su señal de terminacion (`return` de bucle while, termina `run()`), si es el ultimo debe meter tantas pildoras como workers en la cola de la siguiente etapa. Globalmente, el programa termina cuando se llega a la cant maxima de ordenes en `TerminationTracker` (`termination.awaitAll()`) y cuando no hay mas workers vivos (`workers.joinAll()`).

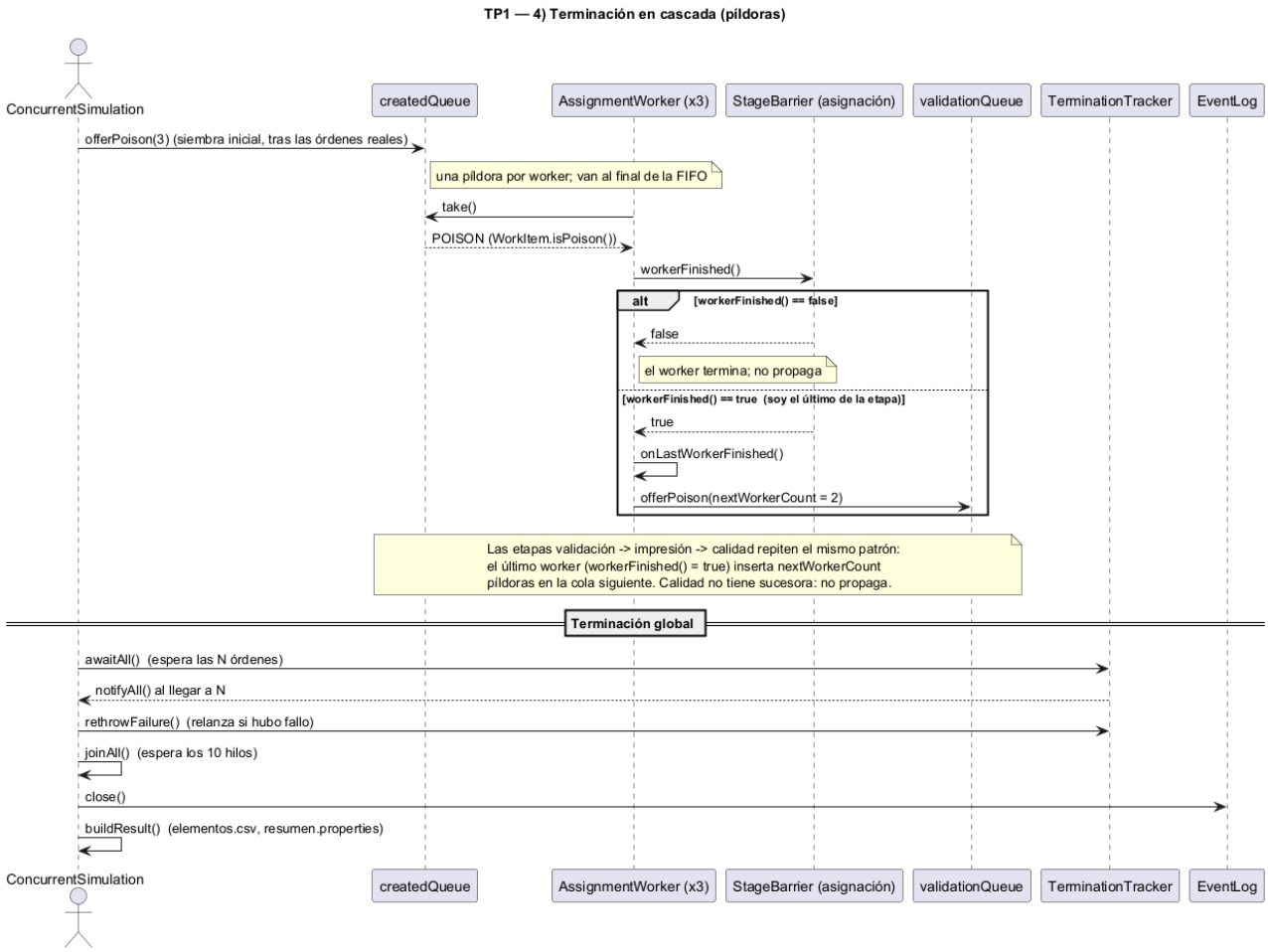



Figure 5: Diagrama de secuencia — terminacion

# 3. Recursos compartidos

Los recursos compartidos, el estado que se protege y su mecanismo de protección son:

| Recurso compartido | Estado protegido | Protección |
|---|---|---|
| Colas entre etapas (`StageQueue`) | Elementos y orden FIFO de la deque | `synchronized` + `Semaphore` |
| Conjunto de impresoras (`PrinterPool`) | Estado y orden asignada de cada impresora; disponibilidad total | `synchronized` + `Semaphore` |
| Registro de eventos (`EventLog`) | Archivo de eventos y contador `sequence` | `synchronized` |
| Terminación (`TerminationTracker`) | Cantidad de órdenes terminadas y fallo informado | `synchronized` + `wait`/`notifyAll` |
| Cierre de etapa (`StageBarrier`) | Cantidad de workers que finalizaron | `synchronized` |
| Arranque (`StartGate`) | Cantidad de workers listos y estado de la compuerta | `synchronized` + `wait`/`notifyAll` |

Se crean cuatro instancias independientes de `StageQueue`, todas con la misma implementación: `createdQueue` alimenta asignación; `validationQueue` conecta asignación con validación; `printingQueue`, validación con impresión; y `qualityQueue`, impresión con control de calidad. En cada frontera cambian el productor y el consumidor, pero la cola siempre conserva el orden FIFO de las órdenes y de las píldoras de cierre.

`Order` y `Printer` no poseen lock propio. Una orden pertenece al único worker que la retiró de su cola, mientras que toda modificación de una impresora se realiza dentro de `PrinterPool`; por ello, la exclusión mutua la

aportan los recursos de la tabla y no los objetos de datos.

# 4. Condiciones de carrera posibles y cómo se evitan

1. **Dos workers de asignación podrían reservar la misma impresora.** Si ambos vieran una impresora libre antes de marcarla, esa impresora quedaría asignada a dos órdenes. El semáforo limita las reservas posibles y `synchronized` hace indivisible la búsqueda y el cambio a `RESERVED`.
2. **Dos workers de una etapa podrían retirar la misma orden.** El semáforo de `StageQueue` entrega un permiso por elemento y la extracción de la deque se realiza dentro de `synchronized`; por eso cada orden queda en manos de un solo consumidor.
3. **Una etapa posterior podría observar una orden antes de que se registre su transición.** Esto produciría un evento fuera de su secuencia lógica. El worker actualiza el estado, registra el evento y solo después encola la orden para la etapa siguiente.
4. **Una orden podría quedar entre dos etapas sin que ninguna la procese.** Al retirarla de la cola, un único worker conserva su propiedad hasta registrarla y encolarla en la siguiente; ningún otro hilo puede intervenir en ese recorrido.
5. **La siguiente etapa podría cerrarse mientras aún queda trabajo por llegar.** Las píldoras respetan la misma FIFO que las órdenes y el último worker, definido por `StageBarrier`, las propaga recién cuando todos los workers de su etapa terminaron de producir.
6. **Una impresora podría liberarse o inhabilitarse dos veces.** Las operaciones de `PrinterPool` se serializan con `synchronized` y verifican que la impresora esté reservada antes de modificarla; una transición inválida se detecta de inmediato.

# 5. Estrategia de sincronización

## 5.1 Colas entre etapas (`StageQueue`)

Cada frontera del pipeline tiene una `StageQueue` con una deque FIFO. El bloque `synchronized` protege la operación sobre esa deque: agregar al final y retirar el primero son atómicas entre productores y consumidores, por lo que el orden de los elementos no se corrompe ni una orden puede retirarse dos veces. El `Semaphore` representa los elementos ya publicados: el productor inserta y luego libera un permiso; el consumidor adquiere un permiso antes de retirar. Así espera trabajo sin espera activa.

Las mismas colas transportan órdenes y píldoras de cierre. Cuando el último worker de una etapa termina, `StageBarrier` hace que solo ese worker inserte las píldoras en la cola siguiente. Al compartir la FIFO, las píldoras quedan detrás de todo el trabajo producido y cada etapa puede terminar sin cerrar prematuramente a la otra.

## 5.2 Pool de impresoras (`PrinterPool`)

`PrinterPool` protege la matriz de impresoras y el número de impresoras disponibles. El `Semaphore` impide iniciar una reserva si no queda ninguna libre. Tras adquirir el permiso, `synchronized` vuelve atómica la búsqueda de una impresora disponible y su transición de `AVAILABLE` a `RESERVED`; así dos workers no pueden elegir la misma. Al liberar, se valida y actualiza el estado antes de devolver el permiso. Si una impresora falla pasa a `OUT_OF_SERVICE` y no recupera el permiso, por lo que no vuelve a utilizarse.

## 5.3 Registro de eventos (`EventLog`)

El contador `sequence` y la escritura de cada fila se realizan en el mismo bloque `synchronized`. Incrementar el contador y registrar la fila forman una única operación lógica: ningún hilo puede obtener un número de secuencia mientras otro todavía está escribiendo su evento. Por eso el archivo conserva una secuencia única y creciente, en el mismo orden en que se escribieron los eventos.

### 5.4 Coordinación de arranque y cierre

- `StartGate` protege el conteo de workers listos y la apertura de la compuerta. Así el último registro y la decisión de iniciar se observan como un estado coherente, y ningún worker procesa antes del arranque conjunto. Esto no es prescindible para el correcto funcionamiento pero se hace para cumplir la consigna al pie de la letra cuando se habla de etapas concurrentes, todas comienzan a la vez.
- `StageBarrier` protege el contador de workers que finalizaron. Incrementarlo y determinar si es el último es atómico, por lo que solo uno propaga las píldoras de cierre a la siguiente etapa.
- `TerminationTracker` protege el contador de órdenes terminales y el primer fallo. El hilo principal espera con `wait` hasta que una de esas condiciones cambie y un worker lo despierta con `notifyAll`.

### 5.5 Por qué no hay locks en `Order`/`Printer`

El diseño respeta *ownership* de un solo hilo por objeto en cada instante: una orden está en una cola o en manos de un único worker, y una impresora solo se modifica dentro de `PrinterPool`. Por eso no se necesitan locks propios en `Order` ni en `Printer`: añadirlos sería redundante y ocultaría dónde está la exclusión mutua.

### 5.6 Justificación de la elección

Se usa una cola por transición, en lugar de un lock global o de `wait/notify` directo sobre las listas. Así cada etapa espera solo su propia entrada, sin bloquear a las demás; además, `StageQueue` reutiliza el mismo protocolo para las cuatro fronteras y para el cierre ordenado mediante píldoras.

## 6. Condiciones de espera y despertar de los hilos

| Hilo | Dónde espera | Condición de espera | Quién lo despierta |
|---|---|---|---|
| Los diez workers | `StartGate.awaitOpen()` | La compuerta no está abierta | `main` con `notifyAll()` al confirmar que todos llegaron |
| Los diez workers | `StageQueue.take()` → `Semaphore.acquire()` | No hay elemento en su cola | Un productor con `release()` tras encolar |
| Workers de asignación | `PrinterPool.reserveFor` → `acquire()` | No hay impresoras disponibles | `release()` al liberar una impresora |
| `main` | `TerminationTracker.awaitAll()` | Aún no terminaron todas las órdenes ni hubo fallo | `orderCompleted()`/`fail()` con `notifyAll()` |
| `main` | `WorkerGroup.joinAll()` | Algún worker sigue vivo | La finalización del worker |

## 7. Mecanismo de terminación

**Píldoras + barrera, en cascada.** Junto con las órdenes reales se encola una píldora (valor centinela) por cada worker consumidor de la cola. Como las píldoras van detrás de las órdenes reales en la misma FIFO, ningún worker las recibe antes de que se haya encolado todo el trabajo. Al consumir su píldora, cada worker registra su cierre en el `StageBarrier` de su etapa; el **último** en llegar es el único que fabrica e inyecta las píldoras de la etapa siguiente. La calidad, última etapa, no inyecta nada.

En paralelo, cada vez que una orden llega a un estado terminal (`APPROVED`, `REJECTED`, `PRINT_FAILED`, `DEFECTIVE`) el worker correspondiente avisa a `TerminationTracker`. El principal está bloqueado en `awaitAll()` y se despierta cuando el contador llega a `totalOrders`; entonces hace `join()` de los diez hilos, cierra el registro de eventos y escribe los archivos de resultado. Esto garantiza que al finalizar no queden órdenes en estados intermedios ni impresoras `RESERVED`.

**Cancelación ante fallos.** Si un worker lanza una excepción o se lo interrumpe, el `WorkerGroup` registra el primer fallo en el `TerminationTracker` e interrumpe a todos los workers; los que estaban bloqueados en un semáforo o una demora despiertan con `InterruptedException` y terminan. El principal espera su cierre y relanza el fallo. No se usa `Thread.stop` ni `System.exit`.

# 8. Determinismo

La decisión de cada etapa se toma exclusivamente con `OutcomeDecider`, cuyos resultados dependen solo de la semilla de configuración y del identificador de la orden. Como cada orden recorre el pipeline exactamente una vez (asignación, validación, impresión y calidad a lo sumo una vez), los estados finales son independientes del orden en que se entrelacen los hilos. No se exige igualdad en la impresora asignada, los contadores individuales ni los tiempos.

# 9. Análisis teórico del tiempo de ejecución

El sistema es un pipeline (línea de ensamblaje). Sea `d_i` la demora por orden de la etapa `i` y `W_i` su cantidad de workers. En régimen estacionario, la etapa `i` procesa una orden cada `d_i / W_i`; el **ciclo del sistema** lo fija la etapa más lenta:

```
ciclo = max_i ( d_i / W_i )
```

El tiempo total aproximado es la latencia de llenado más el ciclo por cada orden restante:

```
makespan ≈ ( Σ_i d_i ) + (N − 1) · ciclo
```

Con la configuración oficial (`N = 500`, `d = 50/120/180/80` ms, `W = 3/2/3/2`):

| Etapa | `d_i` (ms) | `W_i` | `d_i / W_i` (ms) |
|---|---|---|---|
| Asignación | 50 | 3 | 16.7 |
| Validación | 120 | 2 | **60** |
| Impresión | 180 | 3 | **60** |
| Calidad | 80 | 2 | 40 |

Validación e impresión son **cuellos de botella conjuntos** (60 ms por orden). Predicción:

```
makespan ≈ 430 + 499 · 60 = 30 370 ms ≈ 30.4 s
```

**Predicciones del barrido.** Como validación e impresión están empatadas como cuello de botella, agregar un worker a *una sola* de ellas no debería mejorar el tiempo (la otra sigue en 60 ms); agregarlo a *ambas* sí lo reduce. Algunas predicciones:

| Configuración (asig/val/imp/cal) | ciclo teórico | makespan teórico |
|---|---|---|
| 1/1/1/1 | 180 ms | ≈ 90.3 s |
| 2/2/2/2 | 90 ms | ≈ 45.3 s |
| 3/2/3/2 (oficial) | 60 ms | ≈ 30.4 s |
| 3/4/3/2 | 60 ms | ≈ 30.4 s (sin mejora) |
| 3/2/4/2 | 60 ms | ≈ 30.4 s (sin mejora) |
| 3/4/4/2 | 45 ms | ≈ 22.9 s |
| 4/4/4/4 | 45 ms | ≈ 22.9 s |

# 10. Resultados experimentales

## 10.1 Protocolo

- Configuración oficial salvo la variable que se modifique en cada barrido.
- Semilla fija, de modo que los estados finales son idénticos entre corridas y solo varía el tiempo.

- Cinco corridas por variante. Usamos la **mediana** como tiempo representativo: una corrida aislada puede tardar más por tareas del sistema operativo, pero la mediana no se altera tanto como el promedio. También mostramos el mínimo y el máximo para ver cuánto variaron las corridas.

El procedimiento reproducible, los datos crudos y el análisis ampliado se encuentran en la documentación de experimentos.

### 10.2 Corrida oficial

Las cinco corridas produjeron **exactamente la misma distribución de estados finales**, lo que confirma el determinismo derivado de `OutcomeDecider` con semilla fija. Solo varió el tiempo:

| Corrida | durationMillis | approved | rejected | printFailed | defective |
|---|---|---|---|---|---|
| 1 | 30 589 | 359 | 71 | 45 | 25 |
| 2 | 30 574 | 359 | 71 | 45 | 25 |
| 3 | 30 896 | 359 | 71 | 45 | 25 |
| 4 | 31 854 | 359 | 71 | 45 | 25 |
| 5 | 31 899 | 359 | 71 | 45 | 25 |
| **Mediana** | **30 896** | 359 | 71 | 45 | 25 |

La dispersión entre corridas es de aproximadamente 4 % (30 574–31 899 ms).

### 10.3 Comparación teórico vs observado

La mediana observada (30 896 ms) coincide con la predicción teórica (30 370 ms) dentro de aproximadamente un 1.7 %. La diferencia se explica por el arranque de la JVM, la latencia de llenado y vaciado del pipeline, la contención del registro de eventos (sección 12) y el *scheduling* del sistema operativo. La coincidencia valida el modelo de cuello de botella de la sección 9: las etapas gobernantes son validación e impresión, ambas con un ciclo de 60 ms por orden.

### 10.4 Efecto de cambiar la cantidad de hilos

En este experimento se mantienen las demoras oficiales y solo se cambia la cantidad de hilos. El tiempo teórico se calcula con el modelo de la sección 9. El runner `tools/analisis/run-experiments.ps1 -Scenario Threads` genera estas variantes:

| asig/val/imp/cal | cuello de botella | ciclo (ms) | makespan teórico (ms) |
|---|---|---|---|
| 1/1/1/1 | impresión | 180 | ~90 250 |
| 2/2/2/2 | impresión | 90 | ~45 340 |
| 3/2/3/2 (oficial) | validación = impresión | 60 | ~30 370 |
| 3/4/3/2 | impresión | 60 | ~30 370 |
| 3/2/4/2 | validación | 60 | ~30 370 |
| 3/4/4/2 | impresión | 45 | ~22 885 |
| 4/4/4/4 | impresión | 45 | ~22 885 |

Los resultados medidos fueron:

| asig/val/imp/cal | mediana (ms) | teórico (ms) | mín.–máx. (ms) |
|---|---|---|---|
| 1/1/1/1 | 77 808 | ~90 250 | 77 798–77 810 |
| 2/2/2/2 | 39 145 | ~45 340 | 39 131–39 154 |

| asig/val/imp/cal | mediana (ms) | teórico (ms) | mín.–máx. (ms) |
|---|---|---|---|
| 3/2/3/2 (oficial) | 30 561 | ~30 370 | 30 557–30 566 |
| 3/4/3/2 | 26 214 | ~30 370 | 26 201–26 219 |
| 3/2/4/2 | 30 507 | ~30 370 | 30 501–30 516 |
| 3/4/4/2 | 19 800 | ~22 885 | 19 798–19 812 |
| 4/4/4/4 | 19 808 | ~22 885 | 19 801–19 810 |

Al pasar de 1 a 2 hilos por etapa, el tiempo casi se reduce a la mitad. La configuración oficial tarda cerca de 30,5 s. Agregar un hilo solo a impresión (3/2/4/2) prácticamente no cambia nada: validación sigue marcando el ritmo.

En cambio, subir solo validación (3/4/3/2) baja el tiempo a 26,2 s. Esto parece contradecir la predicción simple, pero tiene una explicación: impresión recibe 429 órdenes, no las 500 iniciales, porque 71 se rechazan en validación. Por eso el valor teórico sirve como referencia, aunque no describe por completo los casos donde impresión es la etapa más lenta. Con 3/4/4/2 el tiempo baja a 19,8 s; llevar también asignación y calidad a cuatro hilos no da una mejora visible (19,808 s).

Todas las variantes conservaron la misma distribución final de estados (359 aprobadas, 71 rechazadas, 45 fallidas y 25 defectuosas), por lo que los cambios de concurrencia afectaron el tiempo, no el resultado de negocio.

### 10.5 Efecto de cambiar las demoras

En este caso se usan los hilos oficiales (3/2/3/2) y se multiplican todas las demoras por el mismo factor `k`. Así se ve directamente qué pasa cuando cada etapa tarda más o menos. Variantes del runner (`-Scenario Delays`):

| factor | demoras (ms) | cuello | ciclo (ms) | makespan teórico (ms) |
|---|---|---|---|---|
| ×0 | 0/0/0/0 | — | 0 | ~0 (solo overhead) |
| ×0.5 | 25/60/90/40 | validación = impresión | 30 | ~15 185 |
| ×1 (oficial) | 50/120/180/80 | validación = impresión | 60 | ~30 370 |
| ×2 | 100/240/360/160 | validación = impresión | 120 | ~60 740 |

Los resultados medidos fueron:

| factor | demoras (ms) | mediana (ms) | teórico (ms) | mín.–máx. (ms) |
|---|---|---|---|---|
| ×0 | 0/0/0/0 | 61 | ~0 (solo overhead) | 59–67 |
| ×0.5 | 25/60/90/40 | 15 398 | ~15 185 | 15 376–15 484 |
| ×1 (oficial) | 50/120/180/80 | 30 564 | ~30 370 | 30 557–30 576 |
| ×2 | 100/240/360/160 | 60 931 | ~60 740 | 60 919–60 962 |

Los números siguen una relación casi lineal: con la mitad de demora se tarda aproximadamente la mitad (15,398 s), y al duplicarla se tarda casi el doble (60,931 s). La pequeña diferencia con el valor teórico, de unos 200 ms, es el costo fijo de iniciar la simulación, coordinar hilos y escribir los archivos.

Con demoras nulas, todo el pipeline tarda solo 61 ms. Esto muestra que, en la configuración oficial, casi todo el tiempo corresponde al trabajo simulado y no a la infraestructura concurrente. En todas las variantes se mantuvieron los mismos estados finales: 359/71/45/25.

## 11. Justificación de las decisiones de diseño

El ADR-01 documenta la decisión central: una **cola por transición** de etapa (`StageQueue`: FIFO con `Semaphore` + lock interno) en lugar de un lock global o de `wait/notify` manual.

- **Frente a un lock global (opción A):** un único lock serializaría etapas que no compiten por el mismo dato y no reflejaría que cada etapa es un recurso lógico distinto. La cola por transición permite que una etapa lenta solo haga crecer su propia bandeja de entrada sin bloquear a las demás.
- **Frente a `wait/notifyAll` manual (opción C):** el semáforo encapsula el conteo de "cuántas órdenes esperan" y evita el *thundering herd* de despertar a todos los workers de una etapa por un solo ítem nuevo.
- **Ownership de un solo hilo por objeto:** una orden es tocada por un único hilo a la vez (el que la extrajo de la cola) y una impresora solo se modifica dentro de la sección crítica de `PrinterPool`. Por eso `Order` y `Printer` no llevan lock propio: la exclusión mutua la dan las estructuras compartidas.
- **Píldoras + `StageBarrier` en cascada:** el cierre no se calcula con un número fijo de órdenes por etapa (que depende de `OutcomeDecider`), sino con la cuenta real de workers que terminan; así cada worker cierra la etapa siguiente solo después de haber encolado todo lo suyo.
- **`OutcomeDecider` como única fuente de decisión:** garantiza determinismo para una semilla e id de orden dados y concentra las reglas de negocio fuera de la sincronización.

## 12. Conclusiones

- **Correctitud:** las cuatro etapas se ejecutan concurrentemente y cada orden alcanza exactamente un estado terminal, sin estados intermedios residuales ni impresoras `RESERVED` al finalizar. Las cinco corridas oficiales produjeron la misma distribución de estados (359/71/45/25), confirmando el determinismo del `OutcomeDecider`. La suite de pruebas (pública + del grupo) verifica los invariantes de estados, recursos y terminación.
- **Desempeño:** la corrida oficial y los dos barridos confirman el modelo de pipeline. Al variar hilos, el análisis se refinó para considerar cuántas órdenes llega a procesar cada etapa: validación gobierna la configuración oficial, mientras que impresión procesa menos órdenes por los rechazos previos. Al escalar las demoras, el tiempo resultó lineal y el overhead fijo fue cercano a 200 ms.
- **Aprendizajes:** la combinación de colas FIFO con semáforos, píldoras de cierre y una barrera por etapa resulta en un pipeline simple de razonar, sin espera activa y con cierre seguro en cascada; el determinismo se apoya en usar `OutcomeDecider` como única fuente de decisión de negocio.